# A Self-Stabilizing Min-Max Consensus via Path-loop Detection

Angela Cortecchia[0009−0000−3650−7450], Danilo Pianini[0000−0002−8392−5409], and Mirko Viroli[0000−0003−2702−5702]

Alma Mater Studiorum—Università di Bologna
*Department of Computer Science and Engineering*
Via dell'Università 50, Cesena (FC), 47522, Emilia-Romagna, Italy
{angela.cortecchia, danilo.pianini, mirko.viroli}@unibo.it

**Abstract.** In large-scale distributed systems, such as the Internet of Things (IoT), min-max consensus algorithms provide a mechanism for collective coordination by enabling nodes to converge on a "best" value produced by one of the participants in the computation. However, min-max consensus algorithms are monotonic and non-self-stabilizing by nature: once a value is merged into the aggregate it cannot be retracted, leading to propagation of stale or incorrect data in the presence of transient faults or topology changes. In this work, we propose a novel self-stabilizing min-max consensus algorithm ensuring convergence to the best available value in the network by propagating information along shortest valid paths. Each gossip message carries a value and path of nodes that have acknowledged it, enabling loop-freedom and natural pruning of obsolete contributions. We rely on field-based coordination and specifically the Aggregate Computing paradigm to present the algorithm, prove self-stabilization, and provide an implementation as a reusable library for the *Collektive* DSL. This work contributes a foundational building block for resilient coordination in pervasive computing systems, paving the way to more complex, self-stabilizing distributed applications.

**Keywords:** Aggregate Computing · Gossip Algorithm · Self-Stabilization · Distributed Systems.

## 1 Introduction

Gossip protocols are a widely adopted strategy for coordinating large sets of nodes in distributed systems. By enabling each node to periodically exchange and aggregate information with its neighbors, gossip allows the estimation of collective states in a decentralized, scalable, and fault-tolerant manner. This makes gossip a valuable coordination primitive for emerging application domains such as the Internet of Things (IoT), where devices are deployed across physical space to sense, plan, and act collectively [3, 10, 38].

In these scenarios, the ability of the system to adapt to failures, disconnections, and dynamic changes in topology is crucial. Classical gossip algorithms,

however, suffer from fundamental limitations that make them ill-suited to dynamic environments. In particular, once a value has been merged into the collective state, it is challenging to retract or correct it, especially when that value becomes obsolete.

Typical solutions in the literature involve strategies such as periodic resets of the gossip state [22,33], or using of versioning and timestamps to identify and discard stale information [21, 37], to more refined strategies based on running multiple overlapped gossip instances to avoid abrupt information resets [28].

*Contribution* In this paper, we propose to address this limitation for a specific class of gossip protocols, known as *min-max consensus* protocols [25], where the goal is to propagate the best value in the network according to a shared condition, such as selecting the minimum or maximum value among those contributed by nodes. We do so by proposing a novel *self-stabilizing [18] min-max consensus algorithm* that guarantees convergence to the best value present in the network, even in the face of transient faults. Our solution augments each propagated value with a path of validating nodes, allowing devices to track the flow of information and prevent loops. By enforcing value selection through the shortest valid path, our algorithm maintains correctness, locality, and low overhead, while enabling robust recovery from inconsistent states.

The proposed algorithm does not leverage any central coordination point or global reset mechanism. Instead, it relies solely on local interactions and asynchronous execution, making decisions based on the information received from neighbors over a comparison function which determines whether a value is better than another.

*Structure of the paper* The remainder of this paper is organized as follows.

Section 2 introduces the theoretical and practical context of this work: it revisits classical gossip protocols and their limitations in dynamic environments, recalls the notion of self-stabilization, and related approaches, including epidemic techniques, Conflict-free Replicated Data Type (CRDT) based solutions, and aggregate-computing building blocks. This discussion motivates the need for a fully decentralized and self-stabilizing gossip mechanism.

Section 3 presents the proposed solution. We first describe the intuition behind path-based validation and loop detection, then formalize the local decision process. The Section 3.2 also details an implementation in the *Collektive* framework, showing how the algorithm can be realized as a reusable aggregate operator. The formal proof of self-stabilization is provided in Section 3.3, where we show that the algorithm conforms to the minimizing-share pattern, thus guaranteeing convergence after transient perturbations under standard assumptions.

Section 4 reports the experimental evaluation carried out in simulation, demonstrating both correctness under dynamic topology changes and behavior in terms of convergence and scalability. Finally, Section 5 concludes the paper, summarizing the main contribution and outlining directions for future work.

## 2 Background and Related Work

### 2.1 Gossip Protocols

In a generic gossip protocol, each node exchanges values with its neighbors and aggregates them using an operator $f$, such as minimum, maximum, or set union. This process leads to the emergence of a collective state $v$ from a stream of local inputs $x_\delta$, where $\delta$ is a device.

The merge function $f$ is required to be *idempotent* ($f(a, f(a, b)) = f(a, b)$) and *commutative* ($f(a, b) = f(b, a)$), properties which ensure convergence and resilience to message duplication. The resulting protocols are scalable, redundant, and tolerant to individual node failures.

However, these same properties introduce significant limitations: gossip protocols are inherently *asymmetric* and *information-destroying*. Due to the idempotent nature of the merge operation, a value may be merged multiple times without impacting the aggregate result, making it infeasible to track the exact number of occurrences or retract invalid contributions. For instance, with $f = \min(a, b)$, values can decrease but never increase; similarly, $f = \max(a, b)$ only allows increases. This makes it difficult and costly to remove outdated values once they have influenced the aggregate.

This results in a *non-self-stabilizing* protocol, where the system may fail to recover from transient faults or outdated inputs. If a node starts with an incorrect value, or if its value becomes obsolete, it may continue to influence the collective state indefinitely. This leads to inconsistent behavior across nodes, despite local correctness.

These limitations motivate the need for self-stabilizing gossip protocols, which can autonomously recover from inconsistencies and converge to a correct global state. The notion of self-stabilization, first introduced by Dijkstra [18], refers to the ability of a distributed system to reach a legitimate configuration from any initial state within a finite number of steps, and to remain in that configuration unless perturbed by faults. This is particularly relevant in dynamic systems where nodes may join, leave, or change state unpredictably.

Several approaches have been proposed to address these limitations:

- Identify values for $x_\delta$ with a unique identifier of the source or a timestamp, in such a way to update the old values or remove them and keep the newest ones. This approach is used for gossip algorithms that build indexing or routing data structures, e.g., in peer-to-peer systems [21, 37]. However, most of the devices know only a fragment of the aggregate value $v$, or the size of the exchanging data will be too large, since every value has to be tracked individually.
- Periodically restart the gossip protocol to discard the old values of $x_\delta$ and to reset $v$ [22, 33]. As an advantage, this approach is very lightweight, but can lead to lagging information in the network before the changes are effectively acknowledged and large transients in $v_\delta$ during the restart phase. Moreover, it must be avoided to share old gossip values with the new ones;

otherwise there is the risk of contaminating the new gossip with obsolete values, resulting in no benefit from the restart.
- Run multiple overlapping gossip instances, each with a different identifier, to allow a smooth transition between old and new values [28]. This approach can mitigate the issues of periodic restarts at the price of increased complexity and overhead.

Recent advances in Aggregate Computing (AC) [10] have introduced self-stabilizing building blocks [35] to disseminate, collect [6], and elect leaders [27, 29], which can be combined to implement self-stabilizing gossip protocols via the self-organising coordination regions (SCR) pattern [30]. Although these can be composed to implement self-stabilizing gossip protocols, since a leader is required to coordinate the collection and redistribution of information, they need a coordination point to achieve convergence, which may introduce fragility in dynamic networks [7].

### 2.2 Self-stabilizing Leader Election with Local Agreements

A related line of research concerns self-stabilizing leader election and local agreement protocols, which aim at achieving coordination objectives such as electing a unique leader or reaching agreement on a distinguished value starting from arbitrary initial configurations [19]. These algorithms typically provide strong correctness guarantees, ensuring convergence despite transient faults and asynchronous execution. However, they usually focus on establishing a single global structure, such as a leader or a spanning tree, and often require additional assumptions on the execution model or the maintenance of auxiliary global state.

In contrast, the problem addressed in this work is not to elect or maintain a unique global coordinator, but rather to continuously propagate and update the best available value in the network according to a comparator. Alternative approaches that do not rely on explicit symmetry-breaking phases or global agreement steps preserve the decentralized and local nature of gossip-based interactions.

### 2.3 Epidemic Protocols, TTL/Ageing and CRDTs

Epidemic and gossip-based dissemination protocols often address stale information by introducing explicit versioning mechanisms, such as timestamps, sequence numbers, or time-to-live (TTL) counters [17]. These techniques limit the persistence of obsolete data and reduce long-term contamination of aggregates, but their correctness typically depends on parameter tuning and does not provide self-stabilization in the presence of arbitrary transient faults. In particular, outdated values may still influence the system for unbounded time intervals if aging parameters are poorly calibrated or network conditions change.

Conflict-Free Replicated Data Types (CRDTs) [24] represent another prominent approach to achieving convergence in distributed systems by relying on commutative, associative, and idempotent merge operations. While CRDTs guarantee eventual consistency of replicated state, they are inherently monotonic and

do not support the semantic retraction of previously merged contributions once they become obsolete. In contrast, approaches that explicitly couple value selection with lightweight path-based validation can enable the removal of stale information without relying on global resets or monotonic state growth.

Despite the variety of existing approaches, the closest class of protocols to our setting are selector-based gossip mechanisms, which restrict aggregation to the propagation of a single candidate value.

### 2.4 Min-max Consensus

A particular class of gossip protocols, known as *min-max consensus* protocols, employ a comparison function to determine which values to propagate. Provided an arbitrary type $X$, comparator $f : X^2 \to \{1, 0, -1\}$ provides a selection strategy for values in $X$, such that $f(a, b) = 1$ if $a$ is considered better than $b$ according to the selection criterion, $f(a, b) = 0$ if $a$ and $b$ are considered equivalent, and $f(a, b) = -1$ otherwise. These protocols are particularly useful when nodes need to agree on a single value from a set of candidates, such as selecting the minimum or maximum value, or choosing a leader based on specific criteria. However, restricting gossip protocols to the family of min-max consensus protocols does not inherently guarantee self-stabilization: most protocols typically converge only when starting from a legal configuration, and may fail to recover from transient faults or stale information without explicit stabilization mechanisms.

In this work, we present a lightweight and fully asynchronous min-max consensus protocol that combines a comparator-based selection with local path tracking to achieve self-stabilization. Our algorithm ensures loop-freedom and resilience to faults, while maintaining the decentralized nature and minimal state typical of gossip-based solutions.

### 2.5 Field Calculus and Aggregate Computing

AC [11] is a functional macro-programming paradigm for engineering self-adaptive systems, grounded in the notion of *computational fields* [26], distributed data structures that associate situated devices to values. Building upon this foundation, the Field Calculus (FC) [9] provides higher-order functional abstractions for defining, transforming, and composing such fields, based exclusively on local observations and interactions.

An *AC system* is therefore composed of multiple autonomous agents that interact with their surrounding environment through sensors and actuators, while exchanging information with nearby peers, effectively abstracting away the details of the underlying communication topology. Devices execute a common aggregate program in asynchronous rounds, which repeatedly senses the local environment, computes a new local state based on the current state and the information received from neighbors, and shares the updated state with nearby devices. Within this framework, AC enables the expression of *reusable* self-organizing collective behaviors by means functions.

This modeling approach is suitable for a broad spectrum of application domains, and has been successfully applied to various scenarios, including applications to crowd management [11], disaster detection and response [1], morphogenetic algorithms [16], and swarm behaviors [2].

*Practical Aggregate Computing* Multiple aggregate programming languages have been developed to support the practical implementation of AC systems, including Protelis [32] (a stand-alone Domain-Specific Language (DSL)), Scafi [13] (a DSL embedded in Scala), FCPP [8] (a C++ library), and, more recently, *Collektive*[1] [14], a Kotlin-multiplatform DSL. In the remainder of this paper, we will introduce aggregate programming concepts and the proposed gossip algorithm using *Collektive*, although the same principles and patterns can be applied in any of the aforementioned languages.

*Evolution in time and space* To support the evolution of a computational field over both time and space, AC relies on the `share` construct [4], which models stateful information exchange among neighboring agents. The `share` function takes an initial local value together with an operator that computes over a field of values of the same type, and produces an updated value that is locally returned and propagated to neighbors:

```
fun <T> share(intial: T, op: (Field<T>) -> T): T
```

*Computing distances* As a representative example, `share` can be exploited to implement a self-healing variant of the Bellman–Ford algorithm, used to estimate the hop-distance from the nearest device satisfying a given condition [35]:

```
import Double.POSITIVE_INFINITY as infinity
fun <ID, Type> Aggregate<ID>.bellmanFord(isSource: Boolean): Double =
  share(infinity) { distances: Field<Double> ->
    val minDistance = (distances + 1.0).neighbors.values.min ?: infinity
    if (isSource) 0 else minDistance
  }
```

*Gossiping values* A non-self-stabilizing version of a gossip algorithm can be realized through a structurally similar pattern:

```
fun <ID, T> Aggregate<ID>.gossip<Type>(local: T, combiner: (T, T) -> T) =
    share(local) { it.all.values.reduce(combiner) }
```

Here, `local` represents the value contributed by each device, while `combine` is an associative and commutative operator used to aggregate values across the field. For instance, assuming that `temp()` returns the current temperature reading of a device, the following expression computes the maximum temperature *ever* observed in the network:

[1] `https://collektive.github.io/`

```
val maxTemp = gossip(temp(), Double::max)
```

Similarly, the identifiers of all devices that *ever* joined the system can be collected as follows:

```
val allIds = gossip(setOf(localId), Set::plus)
```

Note that, in both cases, we specified that these algorithms return the maximum temperature and the set of all identifiers *ever* observed in the network; indeed, these simple implementations do not provide any mechanism to remove obsolete values, and the `gossip` construct we obtain this way is inherently *non-self-stabilizing*. This behavior represents a known limitation of classical gossip mechanisms.

*Self-stabilizing building blocks and SCR-gossip* AC provides a catalog of fundamental self-stabilizing primitives [35]. These include mechanisms for information dissemination (encompassing adaptive gradient algorithms such as the Bellman–Ford variant discussed above), for information aggregation and for symmetry breaking (commonly adopted for multi-leader election). A key property of these primitives is that their functional composition preserves self-stabilization guarantees.

By relying on these patterns, a self-stabilizing gossip protocol can be implemented by:

1. electing a leader using a self-stabilizing leader election protocol;
2. propagating a distance information from such leader, building a "potential" field with a single minimum at the leader's position;
3. building a minimum spanning tree rooted at the leader by using the potential field as base gradient, and performing collection and aggregation along such tree, so that every device contributes its value exactly once;
4. broadcasting the aggregated value to the entire network with a second gradient-based dissemination, so that every device receives the same value.

These steps resemble an instance of the more general SCR pattern [30], which consists of a symmetry-breaking phase (multi-leader election), followed by the construction of downstream (from the leader) and upstream (to the leader) communication structures.

Although this solution is conceptually straightforward and self-stabilizing by construction, it can be insufficiently responsive in networks with high churn or frequent topology changes, as the leader election and tree construction phases may take a significant amount of time to converge.

## 3 Self-Stabilizing Min-Max Consensus with Path Loop detection

We now introduce a self-stabilizing min-max consensus algorithm, based on the AC paradigm, that allows nodes to converge to a consistent state even in the presence of transient faults.

### 3.1 Idea and Algorithm

The algorithm is based on the idea of tracking the origin and propagation of the "best" known value in the network (that we denote as $X$) by annotating it with the path of nodes that have validated it, while discarding values whose path is incoherent (loop detection). The algorithm requires, as input, the local value of the node and a comparison function $f$ as described in Section 2.4. Moreover, devices must be uniquely identifiable, and their identifiers must be comparable to break ties. These identifiers can be derived from the hardware (e.g., Medium Access Control (MAC) addresses) or can be software generated (e.g., Universally Unique IDentifiers (UUIDs)). From the algorithm point of view, a change in a device identifier is equivalent to a device being replaced by another one in the network, de-facto increasing the churn. Each device $\delta$ propagates a couple $(x_\delta, P_\delta)$, where $x_\delta$ is the best value found so far according to the comparison function, and $P_\delta$ is the path (ordered list) of nodes that leads to the closest device where $x_\delta = X$.

At every round, each device $\delta$ will receive, for each neighbor $\nu \in \mathcal{N}$, a couple $(x_\nu, P_\nu)$—namely, it will operate on a set of such pairs. The algorithm is a filtering and folding process that proceeds as follows:

1. discard any couple $(x_\nu, P_\nu)$ where $\delta \in P_\nu$ (loop detection);
2. reduce the remaining couples by applying the comparison function $f$ to their $x$ values;
3. in case of a tie, select the couple with the shortest path $P$;
4. in case of further tie, compare the identifiers in $P$ pairwise between the two candidates, and select the one with the lowest[2] identifier at the first position where they differ;
5. if the resulting best couple in the neighborhood ($x_\mathcal{N}$, $P_\mathcal{N}$) is better than the local value, propagate ($x_\mathcal{N}$, $P_\mathcal{N} \cdot \langle\delta\rangle$) and return $x_\mathcal{N}$; otherwise, propagate ($x_\delta$, $\langle\delta\rangle$) and return $x_\delta$.

Figure 1 conveys the idea behind the algorithm graphically by showing colored arcs that represent the path along which the best value (in this case, the minimum identifier) propagates through the network. If the node with the best value (e.g., node 0 in Figure 1) becomes unreachable, the network automatically adapts: the stale information is pruned by the loop detection mechanism, and the next best value (e.g., node 1) is propagated through the network along the shortest valid path.

In memory, this approach has a worst-case cost that grows linearly with the diameter of the network, as the path $P$ can grow up to the diameter in the worst case. To get an idea of the practical implications of this cost, for a large-scale deployment on an Internet-like network, whose router-level diameter is estimated to be in the order of tens (10–40) [12, 20, 23, 39], using UUIDs as identifiers (16 bytes) would require a maximum of 160–640 bytes, which is easily manageable

[2] Lowest here is used for simplicity. As long as a total order is defined on the identifiers, any deterministic tie-breaking strategy can be applied.

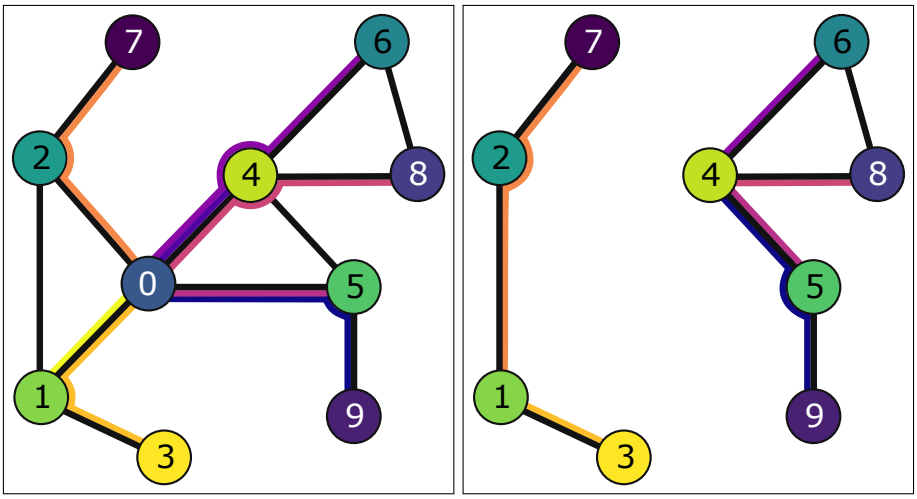


Fig. 1: A graph where nodes represent devices identified by unique IDs, of which we are searching the minimum. Communication links are in black; colored arcs illustrate the shortest valid path used to propagate the best value through the network. The figure on the left shows a stabilized network, while the right one depicts the same network after the failure of the node with the best value (node 0) and the adaptation process.

for most modern networked devices. Of course, smaller identifiers (e.g., 6-byte MAC addresses) can be used to further reduce this overhead.

### 3.2 Implementation in Collektive

Listing 1.1, shows the implementation of the self-stabilizing min-max consensus algorithm in *Collektive*. The code is structured around a compact message format, `GossipValue`, and a single `share`-based state evolution function, `findMaxOf`.

*Message format.* `GossipValue` (Lines 1–6) encodes the pair $(X, P)$ described in Section 3. Property `best` (Line 2) stores the candidate value $X$. Property `path` (Line 3) stores the validation path $P$, namely the sequence of device identifiers that propagated the value. Method `addHop` (Line 5) implements the path extension step by appending the current device identifier when forwarding a candidate to neighbors.

*State evolution via `share`.* Function `findMaxOf` (Lines 8–33) is parametric in the local contribution `local` and in a `Comparator`, so that the same implementation can realize arbitrary selections by passing the appropriate comparator. At each round, the device starts from the locally generated candidate `localGossip`

Listing 1.1: Implementation of the proposed self-stabilizing gossip algorithm proposed inside the DSL *Collektive*.

```
1  data class GossipValue<ID : Comparable<ID>, Value>(
2    val best: Value,
3    val path: List<ID> = emptyList(),
4  ) {
5    fun addHop(id: ID) = GossipValue(best, path + id)
6  }
7
8  fun <ID : Comparable<ID>, Value> Aggregate<ID>.findMaxOf(
9    local: Value,
10   comparator: Comparator<in Value>,
11 ): Value {
12   val localGossip = GossipValue<ID, Value>(best = local)
13   return share(localGossip) { gossip ->
14     gossip.neighbors.values.fold(localGossip) { current, neighbor ->
15       when {
16         localId in neighbor.path -> current // Ignore paths that loop back
17         else -> when(comparator.compare(neighbor.best, current.best)) {
18           0 -> when { // If values tie, select based on path
19             neighbor.path.size == current.path.size ->
20               // If paths are of equal length, compare their last element
21               // (they necessarily come from different neighbors)
22               listOf(neighbor, current).minBy { it.path.last() }
23             // Select the shortest path
24             else -> listOf(neighbor, current).minBy { it.path.size }
25           }
26           // Pick the best value according to the comparator
27           in 1..Int.MAX_VALUE -> neighbor
28           else -> current
29         }
30       }
31     }.addHop(localId)
32   }.best
33 }
```

(Line 12) and executes a `share` (Line 13) over the neighborhood field of received `GossipValue` messages (`gossip`).

The core selection logic is implemented as a fold over neighbors (Line 14), which mirrors the algorithmic steps in Section 3. First, loop-freedom is enforced by discarding any candidate whose path already contains the local identifier (Line 16): such a message would imply that the candidate has traversed a cycle and is therefore considered invalid. Second, candidates whose `best` values tie according to the comparator (Line 18) are ordered by the length of their validation path, preferring the shortest one (Line 24). If the competing paths have equal

length (Line 19), a deterministic tie-breaker is applied based on the last element of the path (Line 22), ensuring stable decisions even under symmetry. In this implementation, we do not need to check the entire same-length paths pairwise for tie-breaking, since, by construction, the `gossip` field will contain one entry per neighbor, and in every case the last element of the path will be the neighbor id that forwarded the candidate to the local device. Additionally, no conflicts are possible with the local candidate, since it is initialized with an empty path, and will win by length against any neighbor-derived candidate with a `best` value that ties with it.

When candidates do not tie, the fold selects the one whose `best` is preferred by the comparator (Line 27). After the best neighbor-derived candidate (or the local one) has been selected, the device acknowledges it by appending its own identifier to the path (Line 31), and finally returns the selected value to the caller via `.best` (Line 32).

*Operational implications.* The only additional information carried beyond classical min/max gossip is the path list, which is used exclusively for (i) loop detection and (ii) deterministic tie-breaking. As a result, stale candidates that survive because of transient faults or topology changes are naturally pruned when they re-enter a cycle, enabling convergence without global resets or coordinated epochs, while preserving a fully local, asynchronous execution model.

### 3.3 Proof of self-stabilization

We prove that the field computed by `findMaxOf` is self-stabilizing under these assumptions: *(i)* each connected component contains a finite set of devices $\mathcal{D}$; *(ii)* each device only considers the last value received from each neighbor (i.e., the neighborhood field contains at most one value per neighbor); *(iii)* device identifiers are totally ordered; and *(iv)* after some time, both local inputs `local` and neighborhood relations stabilize. To make the argument easier to inspect, we separate the proof obligations as follows: *(i) loop-freedom*: forwarded candidates cannot be accepted after re-entering a device already appearing in their path; *(ii) resilience to stale information*: after the environment stabilizes, unsupported candidates are eventually pruned; *(iii) convergence*: the remaining candidates converge to the best value available in each connected component. We then show that the implementation conforms to the minimizing-rep framework of [35, §5.2.2], which gives the self-stabilization result.

**Preliminaries** We recall that the minimizing-rep pattern is defined in field-based coordination (as a generalization of the result in [36]) as follows:

$$\mathrm{rep}(e)\{\, x \mapsto f^R\big(\,\mathrm{minHoodLoc}\,\big(f^{MP}(\mathrm{nbr}\{x\}, \bar{s}),\ s\big),\ x,\ \bar{e}\big)\,\} \tag{1}$$

where:

- minHoodLoc$(\phi, s)$ is a built-in operator returning the minimum of the neighboring field values in $\phi$ and the local self-stabilizing value $s$. If the neighborhood is empty, the result is $s$.
- $f^{MP}$ (the *monotonic-progressive* function) is a stateless function used to "inflate" the neighbors' candidate values before taking the minimum. It must satisfy, w.r.t. a partial order $\leq$:
  1. *Monotonic non-decreasing in the first argument:* if $a_1 \leq a_2$, then $f^{MP}(a_1, \bar{s}) \leq f^{MP}(a_2, \bar{s})$.
  2. *Progressive in the first argument:* $f^{MP}(a, \bar{s}) > a$ or $f^{MP}(a, \bar{s}) = \top$ (where $\top$ is the maximum element of the type).

  In the minimizing-rep pattern, $f^{MP}$ is applied pointwise to the neighbor field nbr$\{x\}$, possibly also depending on additional self-stabilizing inputs $\bar{s}$.
- $f^R$ (the *raising* function) combines the selected minimum candidate, the previous local state, and any extra inputs $\bar{e}$, while being constrained so it cannot prevent convergence. It is required to be *raising* w.r.t. two partial orders (typically: $\leq$ for the "minimum" and another noetherian[3] order $\preceq$ controlling how $f^R$ can deviate transiently), namely:
  1. $f^R(a, a, \bar{e}) = a$;
  2. $f^R(a_1, a_2, \bar{e}) \succeq \min(a_1, a_2)$;
  3. either $f^R(a_1, a_2, \bar{e}) \succ a_2$ or $f^R(a_1, a_2, \bar{e}) = a_1$.

Without loss of generality, we introduce the *minimizing share* pattern, which is a variant of the minimizing-rep pattern, where the share construct [5] is used to replace the combination of rep and nbr:

$$\text{share}(e)\{\, \phi_x \mapsto f^R\big(\,\text{minHoodLoc}\,\big(f^{MP}(\phi_x, \bar{s}),\ s\big),\ \text{local}(\phi_x),\ \bar{e}\big)\,\} \qquad (2)$$

where $\phi_x$ is the neighboring field corresponding to nbr$\{x\}$, and local$(\phi_x)$ is the local device's previous state $x$. This modification speeds up the computation by avoiding sharing the previous round information with neighbors, without affecting the program's self-stabilizing semantics [5].

**Conformance to the minimizing-share pattern** We now show that our implementation is an instance of Equation (2) (minimizing rep pattern), hence, it is self-stabilizing by the results in [35, §5.2.2].

*State space and order.* Let `cmp` be the comparator passed to `findMaxOf`. Assume `cmp` induces a total preorder $\preceq_V$ on `Value` in the standard way:

$$a \preceq_V b \ \triangleq\ \mathsf{cmp}(a, b) \leq 0.$$

Since `findMaxOf` selects the *maximum* according to `Comparator` (Line 27), we define an order where *better values are smaller*:

$$a \leq_V b \ \triangleq\ b \preceq_V a \quad \text{equivalently} \quad a \leq_V b \iff \mathsf{cmp}(a, b) \geq 0.$$

[3] A *noetherian* order is a partial order that contains no infinite ascending chains.

Hence, min w.r.t. $\leq_V$ coincides with max w.r.t. the comparator. The algorithm exchanges and computes elements of $\mathsf{GV} \triangleq \{(v, P) \mid v \in Value,\ P \in ID^*\}$ (corresponding to `GossipValue`). To model loop rejection we extend $\mathsf{GV}$ with the greatest element $\top$ (ignored by minimization), obtaining $\mathsf{GV}_\top \triangleq \mathsf{GV} \cup \{\top\}$.

Let $\leq_{ID}$ be the total order on identifiers. We define a total order $\leq_P$ on paths by *shortest-first* and, in case of equal length, by comparison of the last identifiers in the path (i.e., the sender neighbor, which is guaranteed to be different for different candidates, since the neighborhood field contains at most one entry per neighbor):

$$P_1 \leq_P P_2 \ \triangleq\ \big(|P_1| < |P_2|\big) \ \vee\ \big(|P_1| = |P_2| \wedge P_1\,[|P_1| - 1] \leq_{ID} P_2\,[|P_2| - 1]\,\big),$$

We now define the order $\leq$ over $\mathsf{GV}_\top$ so that the minimum corresponds to the selection policy:

- $\forall g \in \mathsf{GV},\ g \leq \top$;
- for $g_i = (v_i, P_i) \in \mathsf{GV}$:

$$(v_1, P_1) \leq (v_2, P_2) \ \triangleq\ \big(v_1 <_V v_2\big) \ \vee\ \big(v_1 =_V v_2 \wedge P_1 \leq_P P_2\big).$$

*Auxiliary functions.* Fix a device with identifier $\delta$. The sanitizer $\sigma_\delta : \mathsf{GV} \to \mathsf{GV}_\top$ that discards candidates that loop back to $\delta$ can be modeled as a function:

$$\sigma_\delta(v, P) \triangleq \begin{cases} \top & \text{if } \delta \in P, \\ (v, P) & \text{otherwise.} \end{cases}$$

Define the inflation function $f_\delta^{MP} : \mathsf{GV}_\top \to \mathsf{GV}_\top$ by:

$$f_\delta^{MP}(g) \triangleq \begin{cases} \top & \text{if } g = \top, \\ (v,\ P \mathbin{+\!\!+} \langle\delta\rangle) & \text{if } g = (v, P). \end{cases}$$

This corresponds to `addHop(localId)` (Line 31).

The intended role of these two functions is already enough to explain the key properties informally. The sanitizer $\sigma_\delta$ enforces *loop-freedom* locally, because any candidate whose path already contains $\delta$ is mapped to $\top$ and therefore cannot be selected by $\delta$. The inflation function $f_\delta^{MP}$ enforces *progress* by strictly increasing the path whenever a candidate is forwarded. Hence, in a finite stabilized component, a stale candidate that is not regenerated by any local input cannot survive forever: it is either discarded by $\sigma_\delta$ when it closes a loop, or eventually loses to a better/shorter valid candidate.

*Minimizing-share instance.* Consider the following `share` expression that returns a value in $\mathsf{GV}_\top$:

$$\text{share}(e)\{\ \phi \mapsto \text{minHoodLoc}\left(f_\delta^{MP}(\sigma_\delta(\phi)),\ s\right)\ \}, \tag{3}$$

where $\phi$ ranges over the neighbor field entries (i.e., `gossip.neighbors`), $s \triangleq (v_\delta, \langle\delta\rangle)$ is the local self-stabilizing candidate, and $e$ is any initial value. We take $f^R$ to be the identity: $f^R(a, x, \bar{e}) \triangleq a$.

Inside `findMaxOf`, the fold (Line 14) computes the $\leq$-minimum among: *(i)* the local candidate `localGossip` (Line 12) and *(ii)* all neighbor candidates not rejected by the loop guard (Line 16), using the ordering "best first, then shortest path, then smallest sender id" (Lines 18–22). The final `.addHop(localId)` (Line 31) is exactly $f_\delta^{MP}$. Thus, the value returned by the `share` body in the implementation is the same as the value computed by Equation (3) up to the convention that the local candidate is compared with empty path and then inflated. This convention can only affect the `path` chosen when the best values tie ($p$ in $(v, p)$), but never the `best` component ($v$ in $(v, p)$) for any comparison among `GossipValue`s.

*Conditions for minimizing-share.* Under the order $\leq$ above, the assumptions stated at the beginning of the subsection are used as follows. Finiteness of each connected component is needed to rule out infinite strictly progressive path extensions. Keeping only the last value received from each neighbor ensures that the neighborhood field contains a finite set of candidates with at most one entry per sender, which is exactly the setting used by the fold and by the tie-break on the last hop. The total order on identifiers makes the tie-break deterministic. Finally, stabilization of local inputs and neighborhood relations fixes both the set of locally generated candidates and the graph over which valid shortest paths are compared. Under these assumptions:

- *monotonic*: if $g_1 \leq g_2$ then $f_\delta^{MP}(g_1) \leq f_\delta^{MP}(g_2)$, because best values are unchanged and all paths are extended by the same suffix $\langle\delta\rangle$;
- *progressive*: for any $g \in \mathsf{GV}$, $f_\delta^{MP}(g) > g$ since path length strictly increases while best is unchanged.

The identity raising function $f^R(a, x, \bar{e}) = a$ trivially satisfies the raising constraints [35].

Therefore, Equation (3) is an instance of the minimizing-share pattern Equation (2), hence it is self-stabilizing in $\mathsf{GV}_\top$ when the environment stabilizes [5,35]. The function returns the projection $\pi_{\text{best}}(v, P) = v$ (Line 32). Since $\pi_{\text{best}}$ is stateless and the underlying `share` state is self-stabilizing, the returned field of values is self-stabilizing as well; hence, `findMaxOf` is self-stabilizing. □

*Consequences.* The previous argument makes explicit the three properties used in the informal discussion of the algorithm:

1. *loop-freedom* holds because a device never selects a candidate whose path already contains its own identifier.
2. *resilience to stale information* holds because, once local inputs and neighborhood relations stabilize, unsupported candidates are no longer regenerated locally, while progressive path extension cannot continue indefinitely in a finite component without eventually being rejected by the sanitizer.
3. *convergence* holds because the minimizing-share result guarantees stabilization to the $\leq$-minimum candidate, which by construction coincides with the best value according to the comparator, refined by shortest valid path and deterministic identifier-based tie-breaking.

# 4 Evaluation

We now present an evaluation of the proposed self-stabilizing gossip algorithm via simulation, demonstrating its correctness visually, and measuring its convergence time and communication cost.

## 4.1 Experimental setup and reproducibility

We leverage the Alchemist[4] simulator [31], which we also relied upon during the development of the algorithm to perform preliminary tests and debugging. The experiments have been open sourced with a permissive license and are available on a public GitHub repository[5]. An archival copy [15] is available for future reference and reproduction on Zenodo. Every numeric experiment is repeated 32 times with different random seeds. The data presented is the average of these runs.

## 4.2 Scenario description

We configure a bidimensional environment where $N_0$ devices are deployed randomly in a square area of side $L$, and communicate within a radius $R$. Devices are assigned unique identifiers, and run a minimizing operation with a round frequency of $1Hz$, starting asynchronously. We let the system evolve until all devices converge to the same value, and then we perform a sequence of perturbations / topology changes to observe how the network adapts and reconverges to a consistent state.

**Metrics** We evaluate the algorithms correctness by measuring the *Root Mean Squared Error (RMSE)* between the expected gossip value and the actual gossip value as provided by an oracle with global system knowledge which can instantly compute the "correct" value for each device in the network regardless of communication delays. In case of network partitioning, we compute the RMSE separately for each subnetwork, considering the expected gossip value, and then we run a weighted sum of the errors, using the inverse subnetwork size as weight. The global RMSE is defined as follows:

$$\text{RMSE} = \sum_{j \in \mathcal{S}} \frac{1}{\|\mathcal{S}_j\|} \sqrt{\sum_{i \in \mathcal{S}_j} (v_i - \hat{v}_i)^2}$$

where $\mathcal{S}$ is the set of network partitions, $v_i$ is the current value of device $i$, and $\hat{v}_i$ is the true value of device $i$.

To understand the communication cost, we measure the *data rate* required to sustain the algorithm, defined as the average weighted size of messages exchanged

[4] `https://alchemistsimulator.github.io`
[5] `https://bit.ly/3OnjCQA`

per device per time step. We compute the message size by implementing a custom network component which intercepts the calls the framework performs to run network operations. Every message in *Collektive* is a map from an encoding of the location in the Abstract Syntax Tree (AST) that produced the message to the value being sent. In a typical networked deployment, the AST location is hashed to a short identifier; we decided to consider 4 bytes per entry for the key size (which is reasonable for most deployments), while we serialize the corresponding value using Kryo [34].

**Baselines** We compare the proposed algorithm with two baselines: *(i)* a *non–self-stabilizing gossip*, implemented as in Section 2.5, that does not include any mechanism for loop detection, and thus is not self-stabilizing; *(ii)* a *time-replicated gossip*, which is a self-stabilizing gossip algorithm that relies on time-based replication of messages to ensure convergence, as proposed in [35]. The latter requires two parameters: the launch period $p$ and the number of replicas $k$. These are tuned using the relation $p = {}^{4d\tau}/_{(k-1)}$, where $d$ is the network diameter in hops, and $\tau$ is the device frequency. We estimate the network diameter as $d \approx {}^{\sqrt{2L^2}}/_{R} \approx 7$. We use three overlapping replicas ($k = 4$) to balance between convergence time and communication cost; considering that devices run at $1Hz$, the resulting launch period is $p = 9s$.

**The proposed algorithm at work** We show the behavior of the algorithm in Figure 2, in a scenario with $N_0 = 50$, $L = 50m$, and $R = 10m$. The min-max consensus is configured to find the minimum identifier in the network, which is 0 in this case. We can observe that the algorithm self-stabilizes after network partitions and reconnections, as we expect from the theoretical analysis in Section 3.3.

### 4.3 Performance analysis

We analyze the performance of the proposed algorithm in a scenario where $N_0 = 100$, $L = 20m$, and $R = 4m$, where we deploy devices with a uniform random distribution (controlled by the random seed).

At the beginning of the simulation, devices compute a random value in $[-1, 1]$, and the algorithms are configured to find the minimum value in the network. After a stabilization period of $50s$, we introduce a network partition by moving all devices on the right side of the arena in a new arena. $50s$ after the partition, we add a second perturbation, by generating new random values for all devices in the network, this time in $[-{}^1/_2, {}^1/_2]$, and we observe how the different algorithms adapt to the new values in the network. Finally, we move the devices back to the first area, returning to the initial topology, and wait for further $50s$, for a total duration of $200s$.

**Results** In Figure 3, we can observe the Root Mean Square Error (RMSE) over time for the different gossip algorithms after network segmentation, transient

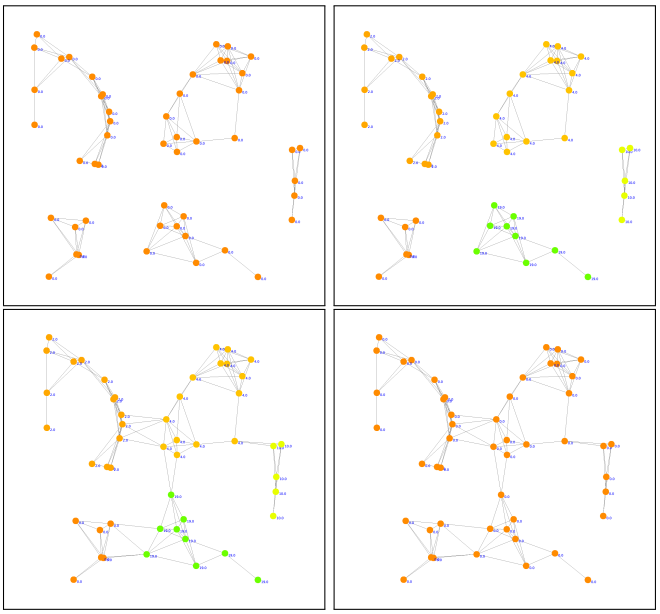

Fig. 2: Self stabilizing nature of the algorithm. Different colors represent different outputs of the min-max consensus algorithm. We partition a stabilized network in five segments (top left). We observe that every segment converges to the best value within the segment (top right). We then reconnect the network (bottom left) and we observe that the whole network returns to the initial best value.

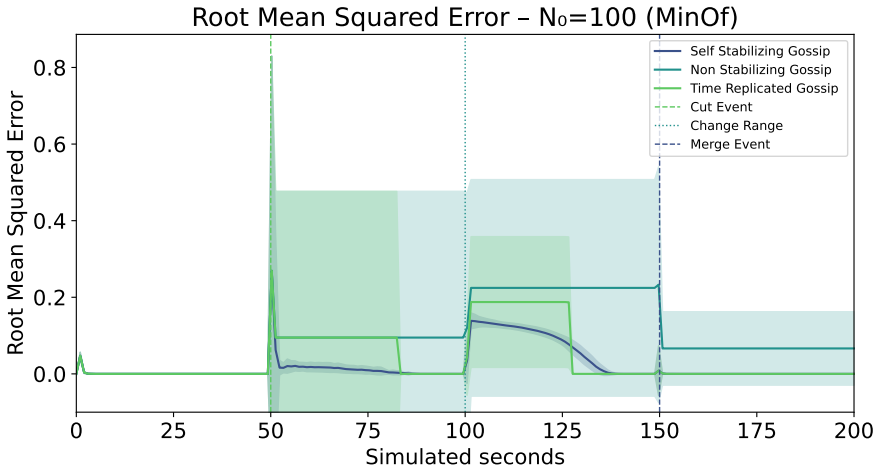


Fig. 3: RMSE over time. Vertical lines show the timing of the disruption events in the scenario, shaded areas show mean $\pm\sigma$ intervals.

fault injection, and network reconnection. The proposed self-stabilizing gossip algorithm shows a different, smoother convergence pattern compared to time-replicated gossip: convergence starts as soon as the perturbation occurs, while time-replicated gossip shows a delayed convergence pattern, with a sudden drop in RMSE after first replica that captured the new values becomes valid. If we consider the time to reach the correct final values after the perturbation, a well-tuned instance of time-replicated gossip outperforms the proposed approach, as it is pure gossip without the overhead of path tracking and loop detection, while the proposed algorithm needs to progressively prune stale candidates through loop detection. However, if the designer prefers a lower mean error during the transient over the time to reach the correct final values, then the proposed algorithm is a better choice, as it starts converging immediately after the perturbation. The other baseline, being non-self-stabilizing, fails to converge after any of the perturbations.

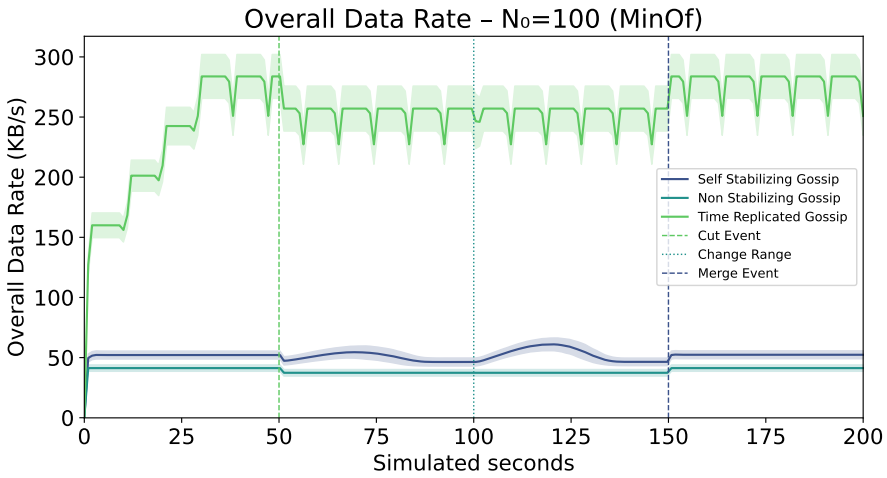


Fig. 4: Communication overhead over time, measured as the overall data rate used by all devices. Vertical lines show the timing of the disruption events in the scenario, shaded areas show mean $\pm\sigma$ intervals.

In Figure 4, we report the data rate over time for the different gossip algorithms under network segmentation, transient fault injection, and subsequent reconnection. The time-replicated gossip algorithm shows a significantly higher data rate than the proposed approach, due to the overhead of maintaining multiple replicas and the associated metadata. We can observe a drop for all the algorithms in the central part of the scenario, when the network is partitioned, as the average neighborhood size decreases. The chart shows the periodic nature of the time-replicated gossip algorithm pretty clearly, with troughs correspond-

ing to the expiration of old replicas, and immediate peaks corresponding to the launch of the new ones. The proposed algorithm requires higher data rate than the non-self-stabilizing gossip algorithm, due to the additional path data, yet the communication overhead is significantly lower than the time-replicated gossip algorithm, whose cost once the algorithms reach regular operation is about four times larger. Upon disruptions, we observe a slow increase in the data rate for the proposed algorithm, as paths grow until they eventually get pruned by loop detection.

## 5 Conclusion and Future Work

In this work, we introduced a self-stabilizing gossip algorithm in the AC paradigm, designed to address key limitations of traditional gossip in dynamic and fault-prone environments. The approach propagates the best value according to a selector function while explicitly encoding the validation path used for its dissemination. By evaluating candidate values individually and enforcing loop-freedom through path tracking, devices adopt only values supported by coherent propagation paths, allowing stale or inconsistent information to be naturally discarded. This preserves the lightweight and fully decentralized nature of gossip without requiring resets, timestamps, or centralized coordination.

We formally showed that the algorithm conforms to the minimizing-share pattern, thereby inheriting self-stabilization guarantees under standard assumptions of eventual input and topology stability.

An implementation in the *Collektive* DSL demonstrates how the approach can be realized as a reusable aggregate operator, and simulation-based evaluation confirms its ability to recover from perturbations and reconverge after topology changes without relying on global resets or centralized coordination.

Overall, the proposed solution provides a building block for resilient distributed coordination, enabling selector-based gossip processes that remain correct and able to adapt to topology changes and perturbations without requiring global coordination.

*Future work* will focus on extending the evaluation of the proposed algorithm to a broader range of network topologies, densities, and fault scenarios, in order to more thoroughly characterize its performance and scalability. In particular, we plan to investigate highly dynamic settings in which nodes frequently join and leave the network, as well as environments affected by heterogeneous communication delays and message loss. Such conditions challenge the eventual-stability assumptions underlying self-stabilization, as the path-based loop detection depends on obsolete paths being gradually eliminated after neighborhood relationships settle. For this reason, we plan to analyze how persistent churn affects convergence behavior and to investigate possible refinements that increase robustness when the system operates under continuously changing conditions.

This loop detection mechanism can be further investigated for other algorithmic contexts beyond gossip-based coordination, as it provides a general approach

to enforce consistency and prevent the propagation of stale information in distributed systems, such as gradient-based coordination.

## References


1. Aguzzi, G., Casadei, R., Pianini, D., Viroli, M.: Dynamic decentralization domains for the internet of things. IEEE Internet Comput. **26**(6), 16–23 (2022). https://doi.org/10.1109/MIC.2022.3216753
2. Aguzzi, G., Casadei, R., Viroli, M.: Macroswarm: A field-based compositional framework for swarm programming. In: Jongmans, S., Lopes, A. (eds.) Coordination Models and Languages - 25th IFIP WG 6.1 International Conference, COORDINATION 2023, Held as Part of the 18th International Federated Conference on Distributed Computing Techniques, DisCoTec 2023, Lisbon, Portugal, June 19-23, 2023, Proceedings. Lecture Notes in Computer Science, vol. 13908, pp. 31–51. Springer (2023). https://doi.org/10.1007/978-3-031-35361-1_2, https://doi.org/10.1007/978-3-031-35361-1_2
3. Apolloni, B., Aitis, C., Maffetti, M.: Watching by gossip. Internet Things **3-4**, 90–103 (2018). https://doi.org/10.1016/J.IOT.2018.09.008, https://doi.org/10.1016/j.iot.2018.09.008
4. Audrito, G., Beal, J., Damiani, F., Pianini, D., Viroli, M.: The share operator for field-based coordination. In: 21st IFIP WG 6.1 International Conference on Coordination Models and Languages (COORDINATION). Lecture Notes in Computer Science, vol. 11533, pp. 54–71. Springer (2019). https://doi.org/10.1007/978-3-030-22397-7_4
5. Audrito, G., Beal, J., Damiani, F., Pianini, D., Viroli, M.: Field-based coordination with the share operator. Log. Methods Comput. Sci. **16**(4) (2020), https://lmcs.episciences.org/6816
6. Audrito, G., Casadei, R., Damiani, F., Pianini, D., Viroli, M.: Optimal resilient distributed data collection in mobile edge environments. Comput. Electr. Eng. **96**(Part), 107580 (2021). https://doi.org/10.1016/J.COMPELECENG.2021.107580, https://doi.org/10.1016/j.compeleceng.2021.107580
7. Audrito, G., Pianini, D., Damiani, F., Viroli, M.: Aggregate centrality measures for iot-based coordination. Sci. Comput. Program. **203**, 102584 (2021). https://doi.org/10.1016/j.scico.2020.102584, https://doi.org/10.1016/j.scico.2020.102584
8. Audrito, G., Torta, G.: FCPP to aggregate them all. Sci. Comput. Program. **231**, 103026 (2024). https://doi.org/10.1016/J.SCICO.2023.103026, https://doi.org/10.1016/j.scico.2023.103026
9. Audrito, G., Viroli, M., Damiani, F., Pianini, D., Beal, J.: A higher-order calculus of computational fields. ACM Trans. Comput. Log. **20**(1), 5:1–5:55 (2019). https://doi.org/10.1145/3285956
10. Beal, J., Pianini, D., Viroli, M.: Aggregate programming for the internet of things. IEEE Computer **48**(9), 22–30 (2015). https://doi.org/10.1109/MC.2015.261, http://dx.doi.org/10.1109/MC.2015.261
11. Beal, J., Pianini, D., Viroli, M.: Aggregate programming for the internet of things. Computer **48**(9), 22–30 (2015). https://doi.org/10.1109/MC.2015.261
12. Carisimo, E., Wang, C., Weaver, M., Bustamante, F.E., Barford, P.: A hop away from everywhere: A view of the intercontinental long-haul infrastructure. Proceedings of the ACM on Measurement and Analysis of Computing Systems **7**(3), 47:1–47:26 (2023). https://doi.org/10.1145/3626778

13. Casadei, R., Viroli, M., Aguzzi, G., Pianini, D.: Scafi: A scala DSL and toolkit for aggregate programming. SoftwareX **20**, 101248 (2022). https://doi.org/10.1016/j.softx.2022.101248, https://doi.org/10.1016/j.softx.2022.101248
14. Cortecchia, A.: Multiplatform self-organizing systems through a kotlin-mp implementation of aggregate computing. In: IEEE International Conference on Autonomic Computing and Self-Organizing Systems, ACSOS 2024 - Companion, Aarhus, Denmark, September 16-20, 2024. pp. 155–157. IEEE (2024). https://doi.org/10.1109/ACSOS-C63493.2024.00048, https://doi.org/10.1109/ACSOS-C63493.2024.00048
15. Cortecchia, A.: angelacorte/experiments-coordination-self- stabilizing-gossip: 1.1.0 (Feb 2026). https://doi.org/10.5281/zenodo.18942476, https://doi.org/10.5281/zenodo.18942476
16. Cortecchia, A., Ciatto, G., Casadei, R., Pianini, D.: Fieldvmc: an asynchronous model and platform for self-organising morphogenesis of artificial structures. Complex & Intelligent Systems **12**(2), 63 (2025). https://doi.org/10.1007/s40747-025-02141-y, https://doi.org/10.1007/s40747-025-02141-y
17. Demers, A.J., Greene, D.H., Hauser, C., Irish, W., Larson, J., Shenker, S., Sturgis, H.E., Swinehart, D.C., Terry, D.B.: Epidemic algorithms for replicated database maintenance. In: Schneider, F.B. (ed.) Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing, Vancouver, British Columbia, Canada, August 10-12, 1987. pp. 1–12. ACM (1987). https://doi.org/10.1145/41840.41841, https://doi.org/10.1145/41840.41841
18. Dijkstra, E.W.: Self-stabilizing systems in spite of distributed control. Commun. ACM **17**(11), 643–644 (1974). https://doi.org/10.1145/361179.361202, https://doi.org/10.1145/361179.361202
19. Dolev, S.: Self-Stabilization. MIT Press (2000), http://www.cs.bgu.ac.il/%7Edolev/book/book.html
20. Faloutsos, M., Faloutsos, P., Faloutsos, C.: On power-law relationships of the internet topology. In: Proceedings of the ACM SIGCOMM 1999 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, August 30 - September 3, 1999, Cambridge, Massachusetts, USA. pp. 251–262. ACM (1999). https://doi.org/10.1145/316188.316229
21. Gupta, I., Birman, K.P., Linga, P., Demers, A.J., van Renesse, R.: Kelips: Building an efficient and stable P2P DHT through increased memory and background overhead. In: Kaashoek, M.F., Stoica, I. (eds.) Peer-to-Peer Systems II, Second International Workshop, IPTPS 2003, Berkeley, CA, USA, February 21-22,2003, Revised Papers. Lecture Notes in Computer Science, vol. 2735, pp. 160–169. Springer (2003). https://doi.org/10.1007/978-3-540-45172-3_15, https://doi.org/10.1007/978-3-540-45172-3_15
22. Jelasity, M., Montresor, A., Babaoglu, Ö.: Gossip-based aggregation in large dynamic networks. ACM Trans. Comput. Syst. **23**(3), 219–252 (2005). https://doi.org/10.1145/1082469.1082470, https://doi.org/10.1145/1082469.1082470
23. Kirci, E.C., Torsiello, V., Vanbever, L.: What is the next hop to more granular routing models? In: Proceedings of the 23rd ACM Workshop on Hot Topics in Networks, HOTNETS 2024, Irvine, CA, USA, November 18-19, 2024. pp. 343–351. ACM (2024). https://doi.org/10.1145/3696348.3696859
24. Letia, M., Preguiça, N.M., Shapiro, M.: Crdts: Consistency without concurrency control. CoRR **abs/0907.0929** (2009), http://arxiv.org/abs/0907.0929
25. Li, H., Yang, J., Yin, Z., Zhou, L., Xi, J., Zheng, Y.: Min-max consensus of multi-agent systems in random networks. Neurocomputing **600**, 128148 (2024).

https://doi.org/10.1016/J.NEUCOM.2024.128148, https://doi.org/10.1016/j.neucom.2024.128148

26. Mamei, M., Zambonelli, F.: Programming pervasive and mobile computing applications: The TOTA approach. ACM Trans. Softw. Eng. Methodol. **18**(4), 15:1–15:56 (2009). https://doi.org/10.1145/1538942.1538945, https://doi.org/10.1145/1538942.1538945
27. Mo, Y., Audrito, G., Dasgupta, S., Beal, J.: Near-optimal knowledge-free resilient leader election. Autom. **146**, 110583 (2022). https://doi.org/10.1016/J.AUTOMATICA.2022.110583, https://doi.org/10.1016/j.automatica.2022.110583
28. Pianini, D., Beal, J., Viroli, M.: Improving gossip dynamics through overlapping replicates. In: Lluch-Lafuente, A., Proença, J. (eds.) Coordination Models and Languages - 18th IFIP WG 6.1 International Conference, COORDINATION 2016, Held as Part of the 11th International Federated Conference on Distributed Computing Techniques, DisCoTec 2016, Heraklion, Crete, Greece, June 6-9, 2016, Proceedings. Lecture Notes in Computer Science, vol. 9686, pp. 192–207. Springer (2016). https://doi.org/10.1007/978-3-319-39519-7_12, https://doi.org/10.1007/978-3-319-39519-7_12
29. Pianini, D., Casadei, R., Viroli, M.: Self-stabilising priority-based multi-leader election and network partitioning. In: IEEE International Conference on Autonomic Computing and Self-Organizing Systems, ACSOS 2022, Virtual, CA, USA, September 19-23, 2022. pp. 81–90. IEEE (2022). https://doi.org/10.1109/ACSOS55765.2022.00026, https://doi.org/10.1109/ACSOS55765.2022.00026
30. Pianini, D., Casadei, R., Viroli, M., Natali, A.: Partitioned integration and coordination via the self-organising coordination regions pattern. Future Gener. Comput. Syst. **114**, 44–68 (2021). https://doi.org/10.1016/j.future.2020.07.032, https://doi.org/10.1016/j.future.2020.07.032
31. Pianini, D., Montagna, S., Viroli, M.: Chemical-oriented simulation of computational systems with ALCHEMIST. J. Simulation **7**(3), 202–215 (2013). https://doi.org/10.1057/JOS.2012.27, https://doi.org/10.1057/jos.2012.27
32. Pianini, D., Viroli, M., Beal, J.: Protelis: practical aggregate programming. In: Proceedings of the 30th Annual ACM Symposium on Applied Computing, Salamanca, Spain, April 13-17, 2015. pp. 1846–1853 (2015). https://doi.org/10.1145/2695664.2695913, http://doi.acm.org/10.1145/2695664.2695913
33. Pruteanu, A., Iyer, V., Dulman, S.: Churndetect: A gossip-based churn estimator for large-scale dynamic networks. In: Jeannot, E., Namyst, R., Roman, J. (eds.) Euro-Par 2011 Parallel Processing - 17th International Conference, Euro-Par 2011, Bordeaux, France, August 29 - September 2, 2011, Proceedings, Part II. Lecture Notes in Computer Science, vol. 6853, pp. 289–301. Springer (2011). https://doi.org/10.1007/978-3-642-23397-5_29, https://doi.org/10.1007/978-3-642-23397-5_29
34. Taranov, K., Bruno, R., Alonso, G., Hoefler, T.: Naos: Serialization-free RDMA networking in java. In: Calciu, I., Kuenning, G. (eds.) Proceedings of the 2021 USENIX Annual Technical Conference, USENIX ATC 2021, July 14-16, 2021. pp. 1–14. USENIX Association (2021), https://www.usenix.org/conference/atc21/presentation/taranov
35. Viroli, M., Audrito, G., Beal, J., Damiani, F., Pianini, D.: Engineering resilient collective adaptive systems by self-stabilisation. ACM Trans. Model. Comput. Simul. **28**(2), 16:1–16:28 (2018). https://doi.org/10.1145/3177774

36. Viroli, M., Damiani, F.: A calculus of self-stabilising computational fields. In: eva Kühn, Pugliese, R. (eds.) Coordination Languages and Models, LNCS, vol. 8459, pp. 163–178. Springer-Verlag (Jun 2014). https://doi.org/10.1007/978-3-662-43376-8_11, proceedings of the 16th Conference on Coordination Models and Languages (Coordination 2014), Berlin (Germany), 3-5 June. Best Paper of Discotec 2014 Federated conference.
37. Voulgaris, S., van Steen, M.: An epidemic protocol for managing routing tables in very large peer-to-peer networks. In: Brunner, M., Keller, A. (eds.) Self-Managing Distributed Systems, 14th IFIP/IEEE International Workshop on Distributed Systems: Operations and Management, DSOM 2003, Heidelberg, Germany, October 20-22, 2003, Proceedings. Lecture Notes in Computer Science, vol. 2867, pp. 41–54. Springer (2003). https://doi.org/10.1007/978-3-540-39671-0_5, https://doi.org/10.1007/978-3-540-39671-0_5
38. Zambonelli, F.: Toward sociotechnical urban superorganisms. Computer **45**(8), 76–78 (2012). https://doi.org/10.1109/MC.2012.280, https://doi.org/10.1109/MC.2012.280
39. Zhu, J., Zhang, Y., Wang, Y., Zhang, H., Fang, B.: A method for quantifying global network topology based on a mathematical model. Mathematics **12**(19), 3114 (2024). https://doi.org/10.3390/math12193114